

Mini Projet - Programmation noyau et système

Olivia Manz, Yoann Archier

Repo github



HES-SO Master

Engineering (MSE) • Computer science

Construction de systèmes embarqués sous Linux • Jacques Supcik

Semestre printemps 2026

Contenu

1	Résumé du laboratoire	3
2	Description de l'application	3
2.1	Module noyau (fanctl.ko)	3
2.2	Daemon userspace (fanctl_daemon)	3
2.3	Application CLI (fanctl_cli)	4
3	Installation et lancement	4
3.1	Prérequis	4
3.2	Compilation et installation	4
3.3	Lancement automatique	4
3.4	Logs	4
4	Travail réalisé	4
4.1	Intégration de l'écran OLED	4
4.2	Développement du module noyau	5
4.3	Développement du daemon utilisateur	5
4.4	Un Makefile pour les gouverner tous	5
5	Choix de conception	6
5.1	Gestion des fréquences	6
5.2	Représentation du mode dans SysFS	6
5.3	Stratégie de rafraîchissement de l'écran	6
5.4	Gestion de la LED Power	6
6	Optimisations et perspectives	7
6.1	Perspectives d'amélioration	7
6.2	Optimisations réalisées	7
7	Difficulté	8
7.1	Mauvais export	8
7.2	strncmp et le sysfs	8
7.3	IPC avec FIFO	8
8	Synthèse sur ce qui a été appris/exercé	9
9	Remarques et choses à retenir	9
10	Feedback personnel sur le laboratoire	10

1 | Résumé du laboratoire

Ce projet est une application permettant de simuler la gestion de la vitesse de rotation d'un ventilateur en fonction de la température du processeur.

2 | Description de l'application

L'application est composée de trois parties principales : un module noyau, un daemon userspace et une application qui fournit une interface utilisateur en ligne de commande.

2.1 Module noyau (`fanctl.ko`)

Le module noyau gère la LED status (`gpio10`) et expose deux attributs via `sysfs` :

- `/sys/class/fanctl/fanctl/frequency` : fréquence de clignotement de la LED, de 1 à 20 Hz
- `/sys/class/fanctl/fanctl/mode` : mode de fonctionnement (**auto** ou **manual**)

En mode **auto**, la fréquence est déterminée par la température du CPU :

- $< 35^{\circ}\text{C} \rightarrow 2 \text{ Hz}$
- $< 40^{\circ}\text{C} \rightarrow 5 \text{ Hz}$
- $< 45^{\circ}\text{C} \rightarrow 10 \text{ Hz}$
- $\geq 45^{\circ}\text{C} \rightarrow 20 \text{ Hz}$

La température est lue toutes les secondes.

En mode **manual**, la fréquence est fixée par l'utilisateur via les boutons ou l'interface IPC.

2.2 Daemon userspace (`fanctl_daemon`)

Le daemon gère :

- La LED power (`gpio362`): clignote lors de chaque appui bouton
- Les boutons *S1*, *S2*, *S3* via interruptions GPIO
- L'affichage sur l'écran OLED
- La communication avec le module via `sysfs`
- L'interface IPC via un socket Unix

L'écran OLED affiche en temps réel :

- La température du CPU (rafraîchie toutes les secondes)
- La fréquence de clignotement de la LED status
- Le mode actuel (auto / manual)

Bouton	Action
S1	Augmente la fréquence de 1 Hz (mode manuel uniquement)
S2	Diminue la fréquence de 1 Hz (mode manuel uniquement)
S3	Bascule entre le mode auto et manuel

En mode auto, les boutons S1 et S2 sont ignorés.

2.3 Application CLI (`fanctl_cli`)

L'application CLI permet de piloter le daemon via un socket Unix (`/tmp/fanctl.sock`).

Commande	Action
<code>fanctl_cli status</code>	Affiche la température, le mode et la fréquence
<code>fanctl_cli mode toggle</code>	Bascule entre le mode auto et manuel
<code>fanctl_cli freq <1-20></code>	Définit la fréquence en Hz (mode manuel uniquement)

Le daemon traite chaque connexion de manière indépendante : il lit la commande, l'exécute et retourne immédiatement une réponse avant de fermer la connexion.

3 | Installation et lancement

3.1 Prérequis

- Le device tree OLED doit être installé sur la cible et la carte redémarrée
- Le rootfs de la cible doit être synchronisé via CIFS sur le host sous `/rootfs/`
- Le script de boot U-Boot doit être **boot.cifs** (voir le laboratoire de préparation « Environnement Linux embarqué »).

3.2 Compilation et installation

Depuis le host, à la racine du projet : **make install**

Cette commande compile tous les composants et les installe sur la cible :

- Module noyau: `/usr/lib/fanctl.ko`
- Daemon: `/usr/bin/fanctl_daemon`
- Application CLI: `/usr/bin/fanctl_cli`
- Script de lancement: `/etc/init.d/S60fanctl`

3.3 Lancement automatique

Le script **S60fanctl** est installé dans `/etc/init.d/` et est exécuté automatiquement au démarrage de la carte. Le module est chargé puis le daemon est lancé en arrière-plan.

3.4 Logs

Les logs du module sont disponibles avec la commande **dmesg**. Les logs du daemon sont disponibles avec la commande **tail -f /var/log/messages**

4 | Travail réalisé

4.1 Intégration de l'écran OLED

L'intégration de l'écran OLED nécessite l'installation d'un Device Tree spécifique sur la cible. Nous utilisons pour cela le projet **oled** fourni. Après compilation, nous transférons le dts comme suit:

```
mount /dev/mmcblk2p1 /mnt/boot
ls /mnt/boot/
cp /mnt/boot/nanopi-neo-plus2.dtb /mnt/boot/nanopi-neo-plus2.dtb.bak
cp /workspace/src/07_miniproj/oled/mydt.dtb /mnt/boot/nanopi-neo-plus2.dtb
umount /mnt/boot
reboot
```

Une fois redémarré, nous testons la présence du endpoint:

```
# ls /dev/i2c-*
/dev/i2c-0
```

Nous testons avec succès le programme de demo `/workspace/src/07_miniproj/oled/oled`. Le texte attendu s'affiche à l'écran.

4.2 Développement du module noyau

Nous commençons la réalisation par le module module noyau en nous inspirons des codelabs précédents dont `/workspace/src/03_drivers/exercice05`. Nous le réalisons par étape: faire clignoter la LED Status (gpio10) à une fréquence fixe, nous exposons ensuite la fréquence sur le sysfs puis le mode. La lecture de la température est testée via `cat /sys/class/thermal/thermal_zone0/temp`.

4.3 Développement du daemon utilisateur

Le daemon userspace doit :

- Lire les boutons S1/S2/S3 via interruptions
- Écrire dans le sysfs pour changer mode/fréquence
- Afficher sur l'écran OLED : mode, température, fréquence

Nous nous basons sur les codelabs différents dont `/workspace/src/04_system/silly/silly_led_control.c` qui gère déjà une led et des boutons.

La lecture de la température (via le sysfs) a déjà été faite dans `/workspace/src/01_environnement/system_calls/syscall.c` On s'en inspire pour non seulement lire la température mais également la fréquence et le mode que nous venons d'exporter.

Nous passons un certain temps à soigner les réponses en cas de mauvaise commande (par exemple essayer de modifier la fréquence alors que le mode n'est pas en manual, renvoyer le mode en cas de mode toggle, etc)

4.4 Un Makefile pour les gouverner tous

Nous créons un makefile qui compilera tous nos binaires en une seule commande et qui les installera sur la carte. Pour ceci, il faut que le rootfs soit bien synchronisé en cifs comme ça a été fait dans les laboratoires précédents. Attention, le rootfs sur le host est situé sous `/rootfs/`. Le module sera installé sous `/usr/lib`, le daemon et l'application sous `/usr/bin` et le script pour lancer le daemon sous `/etc/init.d`

5 | Choix de conception

5.1 Gestion des fréquences

Dans le mode « auto », la fréquence peut prendre les valeurs de 2Hz, 5Hz, 10Hz ou 20Hz. Or, rien n'est indiqué pour le mode manuel. Nous décidons de proposer des pas de 1 pour chaque fréquence avec comme fréquence minimale 1Hz et comme fréquence maximale 20Hz.

Nous n'avons pas implémenté de « mémoire » pour les fréquences manuelles: en passant du mode auto à manuel, la dernière fréquence utilisée en auto sera celle qui sera effective en mode manuel. Ce choix semble cohérent avec l'hypothèse que si il s'agissait de contrôler un moteur, un saut sur les fréquences pourrait être dommageable. En outre, cela simplifie l'implémentation.

5.2 Représentation du mode dans SysFS

Deux approches étaient possibles : exposer une valeur numérique ou exposer une chaîne de caractères.

Bien qu'une représentation numérique simplifie les comparaisons côté logiciel, le choix a été fait d'utiliser les chaînes « auto » et « manual » afin de rendre l'interface SysFS plus lisible et plus intuitive pour l'utilisateur.

5.3 Stratégie de rafraîchissement de l'écran

L'écran fonctionne de telle sorte que on peut mettre à jour uniquement les lignes qui changent.

Le taux de rafraîchissement des données susceptibles de changer (température, fréquence, mode) reste à déterminer. Nous avons pris le parti de mettre à jour ces informations toute les secondes.

Nous savons cependant que certaines données vont changer suite à l'appui sur un bouton:

- la fréquence lors de l'appui sur les boutons pour l'incrémenter et la décrémenter
- le mode et la fréquence lors de l'appui sur le bouton toggle. En effet, en changeant le mode de manuel à automatique, la fréquence sera susceptible de changer.
- la réception d'une commande via l'IPS.

Pour une meilleure réactivité, les éléments susceptibles de changer sont également mis à jour lors de l'appui sur le bouton correspondant. Nous n'avons pas implémenté la mise à jour suite à la réception d'un message via l'IPS, pour éviter les effets de bord en cas de réception de messages répétés.

5.4 Gestion de la LED Power

La led power sert à indiquer à l'utilisateur que l'appui sur le bouton a bien été réalisé. Nous avons choisi de faire clignoter cette led même si l'action retourne une erreur (par exemple modifier la fréquence alors que le mode est en auto). Cela permet à l'utilisateur de voir que le bouton fonctionne, même si l'action désirée n'a pas pu être réalisée.

Le clignotement est actuellement réalisé à l'aide d'un `usleep()`. Cette solution bloque momentanément le traitement principal mais reste acceptable compte tenu des contraintes temporelles limitées du projet.

Une implémentation basée sur un timer supplémentaire ou sur un thread dédié aurait permis une meilleure séparation des responsabilités au prix d'une complexité accrue.

6 Optimisations et perspectives

6.1 Perspectives d'amélioration

- Ajouter une vérification en mode automatique afin d'éviter les changements fréquents de fréquence lorsque la température oscille autour d'un seuil.
- Remplacer l'utilisation de `usleep()` dans le daemon par un mécanisme non bloquant.
- Mettre en place des tests automatisés.
- Gérer les erreurs des appels système (**open**, **write**, **epoll**, etc). Cela n'a pas été réalisé dans les précédents codelab et amène de la complexité et de la lourdeur sur la lisibilité du code mais est indispensable pour un projet en production.
- Dans le daemon nous ouvrons/fermons des fichiers sysfs à chaque appel ce qui génère des syscalls et des accès mémoire dispersés. nous pourrions garder les fd ouverts en permanence, ce qui réduirait les accès mémoire.

6.2 Optimisations réalisées

L'application CLI est très simple et son temps d'exécution dépend principalement de l'utilisation du socket. Nous n'avons pas trouvé de piste d'optimisation.

Concernant le module noyau, nous avons identifié une optimisation pertinente : puisque le timer était partagé entre le clignotement de la led et la lecture de la température, celle-ci était effectuée à chaque tick du timer de clignotement, soit jusqu'à 20 fois par seconde en mode auto. La température du CPU ne variant pas si rapidement, nous avons séparé la logique en deux timers distincts : un timer dédié au clignotement de la LED, et un second timer lisant la température toutes les 5 secondes pour ajuster la fréquence en mode auto. Nous réalisons une courte analyse de la consommation à l'aide de **htop** (Code 1). Puisque notre plateforme est dédiée à cette application, la consommation mémoire et CPU reste largement dans les limites acceptables.

```

0[ *                0.0%] Tasks: 8, 0 thr, 88 kthr; 1 running
1[                0.0%] Load average: 0.00 0.10 0.07
2[*                0.7%] Uptime: 01:21:25
3[                0.0%]
Mem[ |||*@@@       29.8M/474M]
Swp[                0K/0K]
[Main] [I/O]
PID USER      PRI  NI  VIRT   RES   SHR S  CPU%-MEM%  TIME+  Command
265 root        20   0  1876   204   164 S   0.7  0.0  0:14.63 /usr/bin/fanctl
278 root        20   0  3060  2164  1808 R   0.7  0.4  0:00.12 htop
1  root        20   0  2688   324   260 S   0.0  0.1  0:01.76 init
160 root        20   0  2688   316   268 S   0.0  0.1  0:00.01 /sbin/syslogd -
164 root        20   0  2688   316   268 S   0.0  0.1  0:00.01 /sbin/klogd -n
176 root        20   0  3076  2240  1772 S   0.0  0.5  0:00.15 /sbin/mdev -df
248 root        20   0  6156  1924  1404 S   0.0  0.4  0:00.00 sshd: /usr/sbin
252 root        20   0  2688  1908  1740 S   0.0  0.4  0:00.07 -sh

```

Code 1 - Commande Htop

7 | Difficulté

7.1 Mauvais export

Lors de la création du daemon, nous avons par erreur exporté la led gpui10 plutôt que la led de power. Nous avons corrigé notre erreur, relancé le daemon, mais la led reste exportée. Lorsque nous avons rechargé le module, nous obtenions l'erreur

```
# insmod fanctl.ko
insmod: can't insert 'fanctl.ko': Device or resource busy
```

Nous avons pris un moment pour comprendre que l'erreur venait de la led qui était déjà exporté dans le sysfs. Il suffisait alors de la désexporter: `echo 10 > /sys/class/gpio/unexport` mais l'analyse nous a pris du temps. Il nous aurait suffi pourtant de regarder le `dmesg` où l'erreur était explicite

```
[ 4649.442718] fanctl: failed to request gpio 10
```

7.2 strncmp et le sysfs

sysfs ajoute un retour à la ligne en retournant le mode. Lors de la lecture et la comparaison du mode dans le daemon, attention à tronquer la fin de la chaîne de caractère. Cela nous a amené quelques déconvenues lors de la comparaison de chaîne de caractère.

7.3 IPC avec FIFO

Nous avons plusieurs choix pour réaliser l'IPC:

- FIFO: simple à implémenter mais unidirectionnel. Le daemon ne pourra pas répondre à la CLI pour confirmer le message, ou alors il faudra 2 FIFO.
- Pipe: nécessite un fork et nous aimerions que nos 2 processus soient indépendant.
- Message Queue: plus complexe qu'un fifo, mais bidirectionnel. La taille ainsi que le nombre de message doit être déterminé à la création.
- Socket: bidirectionnel mais plus complexe.

Dans le cadre des anciens laboratoires, nous avons réalisé un socket-pair. Nous voulions tester ici une implémentation avec FIFO. Afin de pouvoir envoyer mais aussi recevoir des informations, nous créons 2 FIFO: `/tmp/fanctl_cmd.fifo` et `/tmp/fanctl_resp.fifo`. Les droits sur ces fichiers sont en read et write pour tous.

Le problème que nous rencontrons avec les FIFO est le flush des buffers. Puisque nous sommes parti sur la volonté d'avoir une communication bidirectionnelle, il nous est difficile de gérer et nettoyer 2 fichiers entre les communications. Nous nous retrouvons avec des messages dupliqués, des boucles pour nettoyer, etc. Ce n'est pas idéal. Nous repartons donc sur les sockets heureusement, une bonne partie du code peut être réutilisé.

8 | Synthèse sur ce qui a été appris/exercé

Ce laboratoire nous a permis d'intégrer dans un projet cohérent les différentes briques vues au cours du semestre : développement d'un module noyau avec timers et sysfs, communication entre le noyau et l'espace utilisateur, gestion de périphériques via *GPIO*, et *IPC* entre processus. L'intégration de l'écran OLED a nécessité la modification du *Device Tree* pour exposer le bus *I2C* correspondant, illustrant concrètement le lien entre la description matérielle et les interfaces disponibles en espace utilisateur.

9 | Remarques et choses à retenir

Nous retiendrons que la mise en place d'un projet de cette envergure nécessite une bonne organisation. Il est important de bien comprendre les différentes composants du système et de savoir comment ils interagissent entre eux. Nous avons également vu que les choix de conception faits tôt ont des répercussions sur l'ensemble du développement. Par exemple, exposer le mode en chaîne de caractères (**auto/manual**) plutôt qu'en entier rend l'interface sysfs plus lisible, mais implique un traitement supplémentaire côté daemon lors de la comparaison des valeurs.

Nous avons également vu que la réutilisation de code et de concepts déjà abordés peut grandement faciliter le développement, tout en renforçant notre compréhension globale du système.

10 | Feedback personnel sur le laboratoire

La principale difficulté rencontrée lors de ce laboratoire a été de savoir par où commencer, bien que l'énoncé permettait déjà d'avoir une idée claire du découpage et des interfaces.

Nous avons commencé par tester l'affichage sur l'écran, car un pilote basique était fourni et il est toujours motivant d'obtenir rapidement un résultat visuel.

Les différents laboratoires réalisés au cours du semestre nous ont fourni des briques pour aborder les différentes composantes du système. Nous avons notamment pu réutiliser plusieurs éléments développés précédemment.

Nous avons particulièrement apprécié le fait que ce laboratoire prenne la forme d'un projet. Il permet de mettre en pratique, dans un contexte concret, des notions abordées séparément durant le semestre. Cet aspect synthétique en fait également un excellent exercice de révision.

La principale limite de ce projet réside dans son positionnement en fin d'année académique. La période est chargée en rendus et en préparations d'examens, ce qui réduit le temps disponible pour approfondir certains aspects techniques. Cela est d'autant plus frustrant que le projet se prête naturellement à de nombreuses améliorations qui n'ont pas pu être explorées dans le temps imparti.

Enfin, la réalisation d'un système complet et fonctionnel constitue un aspect particulièrement satisfaisant de ce laboratoire. Le fait de travailler sur une plateforme matérielle utilisée tout au long du semestre renforce le sentiment d'aboutissement, tandis que l'intégration de l'écran OLED apporte une dimension concrète et visuelle appréciable au résultat final.

En conclusion, ce mini-projet nous a semblé bien dimensionné, englobant plusieurs thèmes abordés en cours et intéressant d'un point de vue technique. Malgré un calendrier exigeant, il constitue une excellente synthèse des compétences développées durant le semestre.